Milestone 1: Tackling big data on your laptop

Overall project goal and data

During this course, you will be working on an *individual* project involving big data. The purpose is to get exposure to working with much larger datasets than you have previously in MDS. In particular, you will be building and deploying ensemble machine learning models in the cloud to predict daily rainfall in Australia on a large dataset (~6 GB), where features are outputs of different climate models, and the target is the actual rainfall observation.

You will be using this dataset on figshare. This folder has the output of different climate models as features, and our ultimate goal is to build an ensemble model on these outputs and compare the results with the actual rainfall. At the end of the project, you should have your ML model deployed in the cloud for others to use.

Measurement Helper

This notebook utilized a measurement helper. Decorate any function to measure it:

```
@measure
def combine_csvs(folder):
    ...   # your implementation here

combine_csvs("data/")
```

- **wall** — real elapsed time
- **cpu** — time the processor was actually busy (I/O wait excluded)
- **mem Δ** — change in OS-level Memory (RAM) after the call (RSS)
- we don't have storage I/O estimate here, but when `wall >> cpu`, disk I/O is likely your bottleneck.

In [1]:
```
# Imports
import time, psutil, os, functools
import re
import glob
import zipfile
import time
from pathlib import Path
import json
import requests
import pandas as pd
from urllib.request import urlretrieve
```

In [2]:
```
# Measurement Helper
_proc = psutil.Process(os.getpid())
```

```
def measure(fn):
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        m0 = _proc.memory_info().rss / 1e6
        c0 = time.process_time()
        t0 = time.perf_counter()
        out = fn(*args, **kwargs)
        print(f"{fn.__name__}:  wall={time.perf_counter()-t0:.1f}s  "
              f"cpu={time.process_time()-c0:.1f}s  "
              f"mem Δ{_proc.memory_info().rss/1e6 - m0:+.0f} MB")
        return out
    return wrapper
```

1. Downloading the data

1. Download the data from figshare (look at the URL, and you will see the `article_id` at the end) to your local computer using the figshare API (you need to make use of `requests` library).

2. Extract the zip file, again programmatically, similar to how we did it in class.

> You can download the data and unzip it manually. But we learned about APIs, so we can do it in a reproducible way with the `requests` library, similar to how we did it in class.

> There are 5 files in the figshare repo. The one we want is: `data.zip`

In [3]:
```
# use current notebook directory
WORKDIR = Path.cwd()
OUTPUT_DIR = WORKDIR / "data"
OUTPUT_DIR.mkdir(exist_ok=True)
FORCE_DOWNLOAD = False
```

In [4]:
```
# Necessary metadata
article_id = 14096681
url = f"https://api.figshare.com/v2/articles/{article_id}"
headers = {"Content-Type": "application/json"}
```

In [5]:
```
response = requests.request("GET", url, headers=headers)
data = json.loads(response.text)  # this contains all the articles data, fee
files = data["files"]             # this is just the data about the files, w
files
```

Out[5]:

```
[{'id': 26579150,
  'name': 'daily_rainfall_2014.png',
  'size': 58863,
  'is_link_only': False,
  'download_url': 'https://ndownloader.figshare.com/files/26579150',
  'supplied_md5': 'fd32a2ffde300a31f8d63b1825d47e5e',
  'computed_md5': 'fd32a2ffde300a31f8d63b1825d47e5e',
  'mimetype': 'image/png'},
 {'id': 26579171,
  'name': 'environment.yml',
  'size': 192,
  'is_link_only': False,
  'download_url': 'https://ndownloader.figshare.com/files/26579171',
  'supplied_md5': '060b2020017eed93a1ee7dd8c65b2f34',
  'computed_md5': '060b2020017eed93a1ee7dd8c65b2f34',
  'mimetype': 'text/plain'},
 {'id': 26586554,
  'name': 'README.md',
  'size': 5422,
  'is_link_only': False,
  'download_url': 'https://ndownloader.figshare.com/files/26586554',
  'supplied_md5': '61858c6cc0e6a6d6663a7e4c75bbd88c',
  'computed_md5': '61858c6cc0e6a6d6663a7e4c75bbd88c',
  'mimetype': 'text/x-python'},
 {'id': 26766812,
  'name': 'data.zip',
  'size': 814041183,
  'is_link_only': False,
  'download_url': 'https://ndownloader.figshare.com/files/26766812',
  'supplied_md5': 'b517383f76e77bd03755a63a8ff83ee9',
  'computed_md5': 'b517383f76e77bd03755a63a8ff83ee9',
  'mimetype': 'application/zip'},
 {'id': 26766815,
  'name': 'get_data.py',
  'size': 4113,
  'is_link_only': False,
  'download_url': 'https://ndownloader.figshare.com/files/26766815',
  'supplied_md5': '7829028495fd9dec9680ea013474afa6',
  'computed_md5': '7829028495fd9dec9680ea013474afa6',
  'mimetype': 'text/x-python'}]
```


In [6]:
```python
files_to_dl = ["data.zip"]

for file in files:
    if file["name"] in files_to_dl:
        output_path = OUTPUT_DIR / file["name"]

        if output_path.exists() and not FORCE_DOWNLOAD:
            print(f"{file['name']} already exists. Skipping download.")
        else:
            output_path.parent.mkdir(exist_ok=True)  # ensure folder exists

            print(f"Downloading {file['name']}...")
            urlretrieve(file["download_url"], output_path)
            print(f"Downloaded {file['name']} to {output_path}.")
```

```
Downloading data.zip...
Downloaded data.zip to /Users/jiafuli/Desktop/MDS/Block_6/525/lab/525_ml1/da
ta/data.zip.
```

In [7]:
```python
zip_path = OUTPUT_DIR / "data.zip"
extract_dir = OUTPUT_DIR / "extracted_data"

if not extract_dir.exists():
    print("Extracting files...")
    extract_dir.mkdir(exist_ok=True)
    with zipfile.ZipFile(zip_path, "r") as f:
        f.extractall(extract_dir)
    print("Extraction complete!")
else:
    print("Files already extracted. Skipping extraction.")
```

```
Extracting files...
Extraction complete!
```

In [9]:
```python
%ls -ltr data/extracted_data/
```

```
total 10598424
-rw-r--r--   1 jiafuli  staff   95376895  3 24 10:19 MPI-ESM-1-2-HAM_daily_r
ainfall_NSW.csv
-rw-r--r--   1 jiafuli  staff   94960113  3 24 10:19 AWI-ESM-1-1-LR_daily_ra
infall_NSW.csv
-rw-r--r--   1 jiafuli  staff   82474546  3 24 10:19 NorESM2-LM_daily_rainfa
ll_NSW.csv
-rw-r--r--   1 jiafuli  staff  127613760  3 24 10:19 ACCESS-CM2_daily_rainfa
ll_NSW.csv
-rw-r--r--   1 jiafuli  staff  232118894  3 24 10:19 FGOALS-f3-L_daily_rainf
all_NSW.csv
-rw-r--r--   1 jiafuli  staff  330360682  3 24 10:19 CMCC-CM2-HR4_daily_rain
fall_NSW.csv
-rw-r--r--   1 jiafuli  staff  254009247  3 24 10:19 MRI-ESM2-0_daily_rainfa
ll_NSW.csv
-rw-r--r--   1 jiafuli  staff  235661418  3 24 10:19 GFDL-CM4_daily_rainfall
_NSW.csv
-rw-r--r--   1 jiafuli  staff  294260911  3 24 10:19 BCC-CSM2-MR_daily_rainf
all_NSW.csv
-rw-r--r--   1 jiafuli  staff  295768615  3 24 10:19 EC-Earth3-Veg-LR_daily_
rainfall_NSW.csv
-rw-r--r--   1 jiafuli  staff  328852379  3 24 10:19 CMCC-ESM2_daily_rainfal
l_NSW.csv
-rw-r--r--   1 jiafuli  staff   67784105  3 24 10:19 NESM3_daily_rainfall_NS
W.csv
-rw-r--r--   1 jiafuli  staff   95640682  3 24 10:19 MPI-ESM1-2-LR_daily_rai
nfall_NSW.csv
-rw-r--r--   1 jiafuli  staff  114707410  3 24 10:19 ACCESS-ESM1-5_daily_rai
nfall_NSW.csv
-rw-r--r--   1 jiafuli  staff  116179272  3 24 10:19 FGOALS-g3_daily_rainfal
l_NSW.csv
-rw-r--r--   1 jiafuli  staff  102517965  3 24 10:19 INM-CM4-8_daily_rainfal
l_NSW.csv
-rw-r--r--   1 jiafuli  staff  515458033  3 24 10:19 MPI-ESM1-2-HR_daily_rai
nfall_NSW.csv
-rw-r--r--   1 jiafuli  staff  332813281  3 24 10:19 TaiESM1_daily_rainfall_
NSW.csv
-rw-r--r--   1 jiafuli  staff  337555851  3 24 10:19 NorESM2-MM_daily_rainfa
ll_NSW.csv
-rw-r--r--   1 jiafuli  staff  328787320  3 24 10:19 CMCC-CM2-SR5_daily_rain
fall_NSW.csv
-rw-r--r--   1 jiafuli  staff     952202  3 24 10:19 observed_daily_rainfall
_SYD.csv
-rw-r--r--   1 jiafuli  staff   93829697  3 24 10:19 KIOST-ESM_daily_rainfal
l_NSW.csv
-rw-r--r--   1 jiafuli  staff  102692289  3 24 10:19 INM-CM5-0_daily_rainfal
l_NSW.csv
-rw-r--r--   1 jiafuli  staff  206822938  3 24 10:19 MIROC6_daily_rainfall_N
SW.csv
-rw-r--r--   1 jiafuli  staff   55224437  3 24 10:19 BCC-ESM1_daily_rainfall
_NSW.csv
-rw-r--r--   1 jiafuli  staff  124586961  3 24 10:19 GFDL-ESM4_daily_rainfal
l_NSW.csv
-rw-r--r--   1 jiafuli  staff   46286371  3 24 10:19 CanESM5_daily_rainfall_
NSW.csv
-rw-r--r--   1 jiafuli  staff  333489879  3 24 10:19 SAM0-UNICON_daily_rainf
```

```
all_NSW.csv
drwxr-xr-x  30 jiafuli  staff        960  3 24 10:19 __MACOSX/
```

2. Combining data CSVs

1. Combine data CSVs into a single CSV using pandas.

2. When combining the CSV files, add an extra column called "model" that identifies the model.

   - Tip 1: you can get this column populated from the file name, eg: for file name "SAM0-UNICON_daily_rainfall_NSW.csv", the model name is SAM0-UNICON
   - Tip 2: Remember how we added "year" column when we combined airline CSVs. Here the regex will be to get word before an underscore ie, `"/([^_]*)"`

   > Note: There is a file called `observed_daily_rainfall_SYD.csv` in the data folder that you downloaded. Make sure you exclude this file (programmatically or just take out that file from the folder) before you combine CSVs. We will use this file in our next milestone.

3. Wrap your combine function with `@measure` and ***compare*** wall time, CPU time, and memory across machines. Record results in the table below.

4. Based on the comparison, in 3–5 sentences, reflect on what you observe: how do the numbers differ across machines? What does the `wall` / `cpu` ratio tell you about where the bottleneck is? What surprised you?

The wall to cpu ratio of approximately 1.05 (552.7s vs 524.6s) is quite revealing. This tight ratio tells us that the bottleneck is heavily CPU-bound rather than I/O-bound, meaning the SSD handles file reading efficiently, but the 1.4 GHz processor is working at maximum capacity to parse and concatenate the dataframes. What is most surprising is the massive memory delta of +1690 MB, which consumes a dangerously large chunk of an 8 GB RAM system and perfectly illustrates why raw Pandas is extremely inefficient for Big Data tasks.

Member	Approach	OS	RAM	Processor	SSD?	wall (s)	cpu (s)	mem Δ (MB)
1 (Yusheng Li)	combine	macOS Sequoia 15.7.3	8 GB	1.4 GHz Quad-core Intel Core i5	Yes	552.7	524.6	+1690

In [11]:
```
data_dir = Path("data/extracted_data")

@measure
def combine_csvs(folder_path):
    files = list(folder_path.glob("*.csv"))

    combined_df = pd.concat(
```

```
        (
            pd.read_csv(file)
            .assign(model=re.findall(r"([^_]+)", file.name)[0])
            for file in files
            if file.name != "observed_daily_rainfall_SYD.csv"
        ),
        ignore_index=True
    )

    output_path = folder_path.parent / "combined_data.csv"
    combined_df.to_csv(output_path, index=False)

    print(f"Combined data saved to: {output_path}")
    print(f"Total length: {len(combined_df)}")
    return combined_df

df_combined = combine_csvs(data_dir)
```

```
Combined data saved to: data/combined_data.csv
Total length: 62467843
combine_csvs:  wall=552.7s  cpu=524.6s  mem Δ+1690 MB
```

In [12]:
```
df_combined.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 62467843 entries, 0 to 62467842
Data columns (total 7 columns):
 #   Column         Dtype
---  ------         -----
 0   time           object
 1   lat_min        float64
 2   lat_max        float64
 3   lon_min        float64
 4   lon_max        float64
 5   rain (mm/day)  float64
 6   model          object
dtypes: float64(5), object(2)
memory usage: 3.3+ GB
```

In [13]:
```
df_combined.head()
```

Out[13]:

	time	lat_min	lat_max	lon_min	lon_max	rain (mm/day)	model
0	1889-01-01 12:00:00	-35.439867	-33.574619	141.5625	143.4375	4.244226e-13	MPI-ESM-1-2-HAM
1	1889-01-02 12:00:00	-35.439867	-33.574619	141.5625	143.4375	4.217326e-13	MPI-ESM-1-2-HAM
2	1889-01-03 12:00:00	-35.439867	-33.574619	141.5625	143.4375	4.498125e-13	MPI-ESM-1-2-HAM
3	1889-01-04 12:00:00	-35.439867	-33.574619	141.5625	143.4375	4.251282e-13	MPI-ESM-1-2-HAM
4	1889-01-05 12:00:00	-35.439867	-33.574619	141.5625	143.4375	4.270161e-13	MPI-ESM-1-2-HAM

3. Load the combined CSV to memory and perform a simple EDA

rubric={correctness:10,reasoning:10}

1. Investigate at least two of the following approaches to reduce memory usage while performing the EDA (e.g., value_counts).
   - Changing `dtype` of your data
   - Load just columns that we want
   - Loading in chunks
2. Wrap each approach with `@measure` and ***compare*** wall time, CPU time, and memory across machines. Record results in the table below.
3. Based on the comparison, in 3–5 sentences, reflect on what you observe: which approach was most effective at reducing memory? Did reducing memory come at a cost to wall time? Which approach would you choose in practice and why?

In [17]:

```
# Changing dtype of the data
file_path = "data/combined_data.csv"

@measure
def eda_dtypes(path):
    print("Approach 1: Changing dtype")
    optimized_dtypes = {
        'lat_min': 'float32',
        'lat_max': 'float32',
        'lon_min': 'float32',
        'lon_max': 'float32',
```

```
        'rain (mm/day)': 'float32',
        'model': 'category'
    }
    df = pd.read_csv(path, dtype=optimized_dtypes)
    result = df.groupby('model', observed=True)['rain (mm/day)'].mean()
    return result

res_dtypes = eda_dtypes(file_path)
res_dtypes.head()
```

```
Approach 1: Changing dtype
eda_dtypes:  wall=119.3s  cpu=95.9s  mem Δ+166 MB
```

Out[17]:

```
model
MPI-ESM-1-2-HAM    1.610720
AWI-ESM-1-1-LR     2.026071
NorESM2-LM         2.230799
ACCESS-CM2         1.787025
FGOALS-f3-L        1.627373
Name: rain (mm/day), dtype: float32
```

In [18]:

```
# load just columns that we want
@measure
def eda_usecols(path):
    print("Approach 2: Load necessary columns only")
    df = pd.read_csv(path, usecols=['rain (mm/day)', 'model'])
    result = df.groupby('model')['rain (mm/day)'].mean()
    return result

res_usecols = eda_usecols(file_path)
res_usecols.head()
```

```
Approach 2: Load necessary columns only
eda_usecols:  wall=53.5s  cpu=49.6s  mem Δ-170 MB
```

Out[18]:

```
model
ACCESS-CM2        1.787025
ACCESS-ESM1-5     2.217501
AWI-ESM-1-1-LR    2.026071
BCC-CSM2-MR       1.951832
BCC-ESM1          1.811032
Name: rain (mm/day), dtype: float64
```

In [19]:

```
@measure
def eda_chunking_mean(path, chunksize=1_000_000):
    print("Approach 3: Loading in chunks")

    total_sum = pd.Series(dtype='float64')
    total_count = pd.Series(dtype='float64')

    chunk_iterator = pd.read_csv(
        path,
        usecols=['model', 'rain (mm/day)'],
        chunksize=chunksize
    )

    for chunk in chunk_iterator:
        chunk_sum = chunk.groupby('model')['rain (mm/day)'].sum()
```

```
        chunk_count = chunk.groupby('model')['rain (mm/day)'].count()

        total_sum = total_sum.add(chunk_sum, fill_value=0)
        total_count = total_count.add(chunk_count, fill_value=0)

    overall_mean = total_sum / total_count

    return overall_mean

res_chunking_mean = eda_chunking_mean(file_path)
res_chunking_mean.head()
```

```
Approach 3: Loading in chunks
eda_chunking_mean:  wall=52.7s  cpu=51.0s  mem Δ-36 MB
```

Out[19]:

```
model
ACCESS-CM2        1.787025
ACCESS-ESM1-5     2.217501
AWI-ESM-1-1-LR    2.026071
BCC-CSM2-MR       1.951832
BCC-ESM1          1.811032
dtype: float64
```

Member	Approach	OS	RAM	Processor	SSD?	wall (s)	cpu (s)	mem Δ (MB)
1 (Yusheng Li)	combine	macOS Sequoia 15.7.3	8 GB	1.4 GHz Quad-core Intel Core i5	Yes	552.7	524.6	+1690
1 (Yusheng Li)	approach 1 (dtype)	macOS Sequoia 15.7.3	8 GB	1.4 GHz Quad-core Intel Core i5	Yes	119.3	95.9	+166
1 (Yusheng Li)	approach 2 (usecols)	macOS Sequoia 15.7.3	8 GB	1.4 GHz Quad-core Intel Core i5	Yes	53.5	49.6	-170
1 (Yusheng Li)	approach 3 (chunking)	macOS Sequoia 15.7.3	8 GB	1.4 GHz Quad-core Intel Core i5	Yes	52.7	51.0	-36

Comparing the three approaches, `usecols` and `chunking` were the most effective at reducing memory, surprisingly resulting in negative memory deltas (-170 MB and -36 MB, respectively) as their minimal footprints allowed Python's garbage collector to free up previously used memory. Importantly, this massive memory reduction did not come at a cost to wall time; in fact, bypassing unnecessary column loads slashed execution times from 119.3s (`dtype` method) to around 53s. In practice, I would choose the `chunking` approach combined with `usecols`, as it provides the ultimate out-of-core scalability for datasets that vastly exceed system RAM while maintaining mathematically rigorous aggregations.

3* (Optional) Redo Task 3 with DuckDB

Use DuckDB to run the same EDA query directly on the combined CSV, without loading pandas. Materialize resulting data frame.

```
import duckdb

@measure
def duckdb_eda():
    return duckdb.sql("""
        ...
    """).df()

result = duckdb_eda()
```

Compare the `@measure` output with your best pandas result from Task 3. In 3–5 sentences, explain: why is DuckDB's memory delta so much smaller? What does the `wall` / `cpu` ratio tell you compared to the pandas approaches? When would you prefer DuckDB over pandas for this kind of task?

In [20]:

```
import duckdb

@measure
def duckdb_eda():
    return duckdb.sql("""
        SELECT
            model,
            AVG("rain (mm/day)") AS avg_rain
        FROM 'data/combined_data.csv'
        GROUP BY model
        ORDER BY avg_rain DESC
    """).df()

result = duckdb_eda()
result.head()
```

```
FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(ba
r_color='black'))
duckdb_eda:  wall=11.6s  cpu=54.7s  mem Δ+43 MB
```

Out[20]:

	model	avg_rain
0	INM-CM4-8	2.811463
1	INM-CM5-0	2.669012
2	CMCC-CM2-SR5	2.383389
3	MIROC6	2.301662
4	CMCC-CM2-HR4	2.279350

Member	Approach	OS	RAM	Processor	SSD?	wall (s)	cpu (s)	mem Δ (MB)
1 (Yusheng	combine	macOS Sequoia	8 GB	1.4 GHz Quad-core Intel Core	Yes	552.7	524.6	+1690

Member	Approach	OS	RAM	Processor	SSD?	wall (s)	cpu (s)	mem Δ (MB)
Li)		15.7.3		i5				
1 (Yusheng Li)	approach 1 (dtype)	macOS Sequoia 15.7.3	8 GB	1.4 GHz Quad-core Intel Core i5	Yes	119.3	95.9	+166
1 (Yusheng Li)	approach 2 (usecols)	macOS Sequoia 15.7.3	8 GB	1.4 GHz Quad-core Intel Core i5	Yes	53.5	49.6	-170
1 (Yusheng Li)	approach 3 (chunking)	macOS Sequoia 15.7.3	8 GB	1.4 GHz Quad-core Intel Core i5	Yes	52.7	51.0	-36
1 (Yusheng Li)	DuckDB	macOS Sequoia 15.7.3	8 GB	1.4 GHz Quad-core Intel Core i5	Yes	11.6	54.7	+43
2 (Junli Liu)	baseline	Windows 10	33.8 GB	AMD64 / Intel64 Family 6 Model 170 Stepping 4, GenuineIntel	YES	60.1s	58.4s	+19 MB
2 (Junli Liu)	dtype	Windows 10	33.8 GB	AMD64 / Intel64 Family 6 Model 170 Stepping 4, GenuineIntel	YES	54.6s	53.4s	+58 MB
2 (Junli Liu)	usecols	Windows 10	33.8 GB	AMD64 / Intel64 Family 6 Model 170 Stepping 4, GenuineIntel	YES	39.5s	38.4s	-3 MB
3 (Shuhang Li)	approach 1 (dtype)	macOS	8 GB	Apple M3	Yes	30.7	29.3	+86
3 (Shuhang Li)	approach 2 (select cols)	macOS	8 GB	Apple M3	Yes	15.5	14.9	+168
3 (Shuhang Li)	approach 3 (duckdb)	macOS	8 GB	Apple M3	Yes	2.8	12.3	+278

Compared to the best Pandas approach, DuckDB achieved a drastically faster wall time of 11.6 seconds while maintaining a minimal memory footprint of +43 MB. DuckDB's memory delta remains extremely low because it uses an out-of-core, vectorized execution engine that streams and aggregates data directly from disk without loading full columns into RAM. Furthermore, its cpu time (54.7s) significantly exceeds its wall time (11.6s), indicating that it aggressively leverages multi-threading across multiple CPU cores, completely outperforming Pandas' single-threaded limitation. I would

strongly prefer DuckDB for tasks involving datasets that exceed system memory, as it delivers blazing-fast SQL analytics without the overhead of writing manual chunking logic.

Hardware specifications strictly dictate raw Pandas performance. Systems equipped with expansive RAM effortlessly handle unoptimized, massive data loads without triggering OS-level disk swapping, completing baseline operations in a fraction of the time required by constrained 8 GB environments. Furthermore, modern silicon architectures drastically accelerate single-threaded Pandas operations; advanced processors can execute techniques like column pruning up to 70% faster than older-generation quad-core chips.

The synergy between hardware and parallelized engines amplifies these performance gaps even further. DuckDB universally exhibits a high cpu to wall time ratio across all tested machines, confirming its aggressive and consistent multi-core utilization regardless of the operating system. However, cutting-edge processors process these concurrent data vectors with staggering efficiency, plummeting out-of-core query wall times to under three seconds. Ultimately, while software strategies like column selection are essential survival tactics for memory-limited machines, pairing a vectorized OLAP engine with a modern multi-core processor unlocks transformative performance for Big Data analytics.

4. Perform a simple EDA in R

1. Choose one of the methods listed below for transferring the dataframe (i.e., the entire dataset) from Python to R, and explain why you opted for this approach instead of the others.
   - Parquet file
   - Pandas exchange
   - Arrow exchange
2. Once you have the dataframe in R, perform a simple EDA.

> Note: If you encounter issues running both R and Python in a single notebook, or if you are unable to get rpy2 to work, you may consider alternative approaches. For example, you can save your data in a preferred format and then read it in RStudio (ensure that the required packages are installed manually). You can then copy the relevant R code back into the notebook and comment it out so that the notebook runs smoothly.

In [21]:
```
output_parquet = "data/combined_data.parquet"

pd.read_csv("data/combined_data.csv", usecols=['model', 'rain (mm/day)']).to
```

In [23]:
```
%load_ext rpy2.ipython
```

In [25]:

```
%%R
library(arrow)
library(dplyr)

r_df <- read_parquet("data/combined_data.parquet")

glimpse(r_df)

eda_result <- r_df |>
  group_by(model) |>
  summarize(mean_rain = mean(`rain (mm/day)`, na.rm = TRUE)) |>
  arrange(desc(mean_rain))

print(eda_result)
```

```
Rows: 62,467,843
Columns: 2
$ `rain (mm/day)` <dbl> 4.244226e-13, 4.217326e-13, 4.498125e-13, 4.251282e-
13…
$ model           <chr> "MPI-ESM-1-2-HAM", "MPI-ESM-1-2-HAM", "MPI-ESM-1-2-H
AM…
# A tibble: 27 × 2
   model         mean_rain
   <chr>             <dbl>
 1 INM-CM4-8          2.81
 2 INM-CM5-0          2.67
 3 CMCC-CM2-SR5       2.38
 4 MIROC6             2.30
 5 CMCC-CM2-HR4       2.28
 6 CMCC-ESM2          2.27
 7 NorESM2-MM         2.23
 8 NorESM2-LM         2.23
 9 TaiESM1            2.22
10 ACCESS-ESM1-5      2.22
# ℹ 17 more rows
# ℹ Use `print(n = ...)` to see more rows
```